

Vulnerability Assessment Report

OWASP Juice Shop

1. Executive Summary

This security assessment was conducted on the **OWASP Juice Shop** web application to evaluate its security posture and identify common web application vulnerabilities. The assessment was performed using **Nmap**, **OWASP ZAP (Passive Scan)**, and **Browser Developer Tools (Chrome DevTools)** following ethical and passive security testing practices.

The assessment focused on identifying security misconfigurations, missing security headers, information disclosure, and other common web application weaknesses. The findings were classified based on their potential impact, and practical remediation recommendations were provided.

2. Scan Results

Nmap

Nmap was used to perform network reconnaissance on the target web application by identifying accessible services and gathering basic information about the exposed network.

- Performed host discovery and service detection.
- Identified publicly accessible HTTP and HTTPS services.
- Collected information about exposed network services.
- Executed service version detection and safe NSE scripts to gather additional information.
- **Command used:**

1. `nmap -sV juice-shop.herokuapp.com`
2. `nmap -A --script vuln juice-shop.herokuapp.com`

Evidence:

Figure 1: Nmap -sV scan showing open ports and detected services.

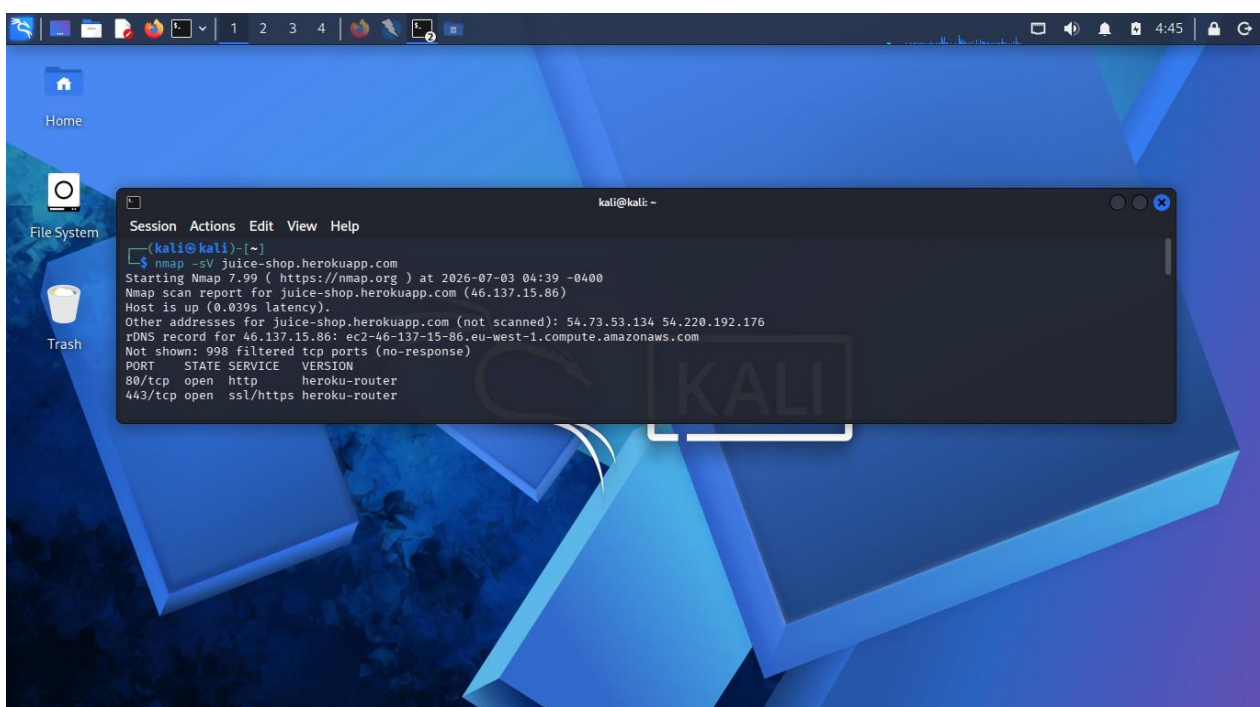
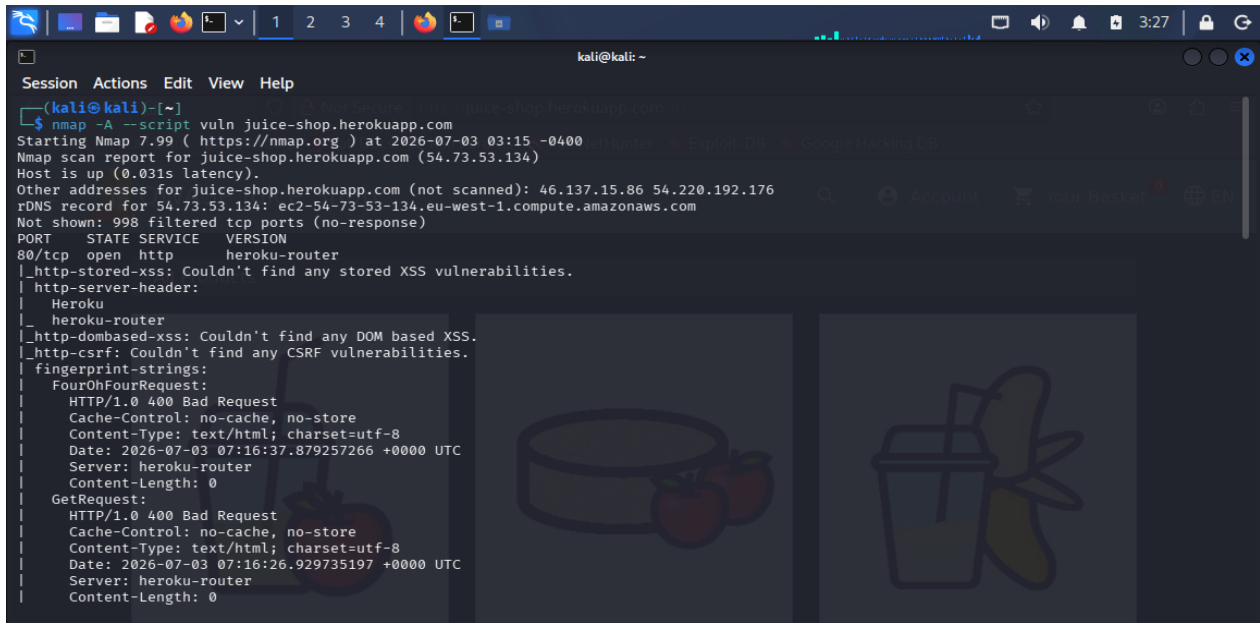


Figure 2: Nmap -A --script scan showing advanced detection and NSE script results.



```
kali@kali: ~  
$ nmap -A --script vuln juice-shop.herokuapp.com  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-03 03:15 -0400  
Nmap scan report for juice-shop.herokuapp.com (54.73.53.134)  
Host is up (0.031s latency).  
Other addresses for juice-shop.herokuapp.com (not scanned): 46.137.15.86 54.220.192.176  
rDNS record for 54.73.53.134: ec2-54-73-53-134.eu-west-1.compute.amazonaws.com  
Not shown: 998 filtered tcp ports (no-response)  
PORT      STATE SERVICE      VERSION  
80/tcp    open  http         heroku-router  
_http-stored-xss: Couldn't find any stored XSS vulnerabilities.  
_http-server-header:  
  Heroku  
  heroku-router  
_http-dombased-xss: Couldn't find any DOM based XSS.  
_http-csrf: Couldn't find any CSRF vulnerabilities.  
fingerprint-strings:  
  FourOhFourRequest:  
    HTTP/1.0 400 Bad Request  
    Cache-Control: no-cache, no-store  
    Content-Type: text/html; charset=utf-8  
    Date: 2026-07-03 07:16:37.879257266 +0000 UTC  
    Server: heroku-router  
    Content-Length: 0  
  GetRequest:  
    HTTP/1.0 400 Bad Request  
    Cache-Control: no-cache, no-store  
    Content-Type: text/html; charset=utf-8  
    Date: 2026-07-03 07:16:26.929735197 +0000 UTC  
    Server: heroku-router  
    Content-Length: 0
```

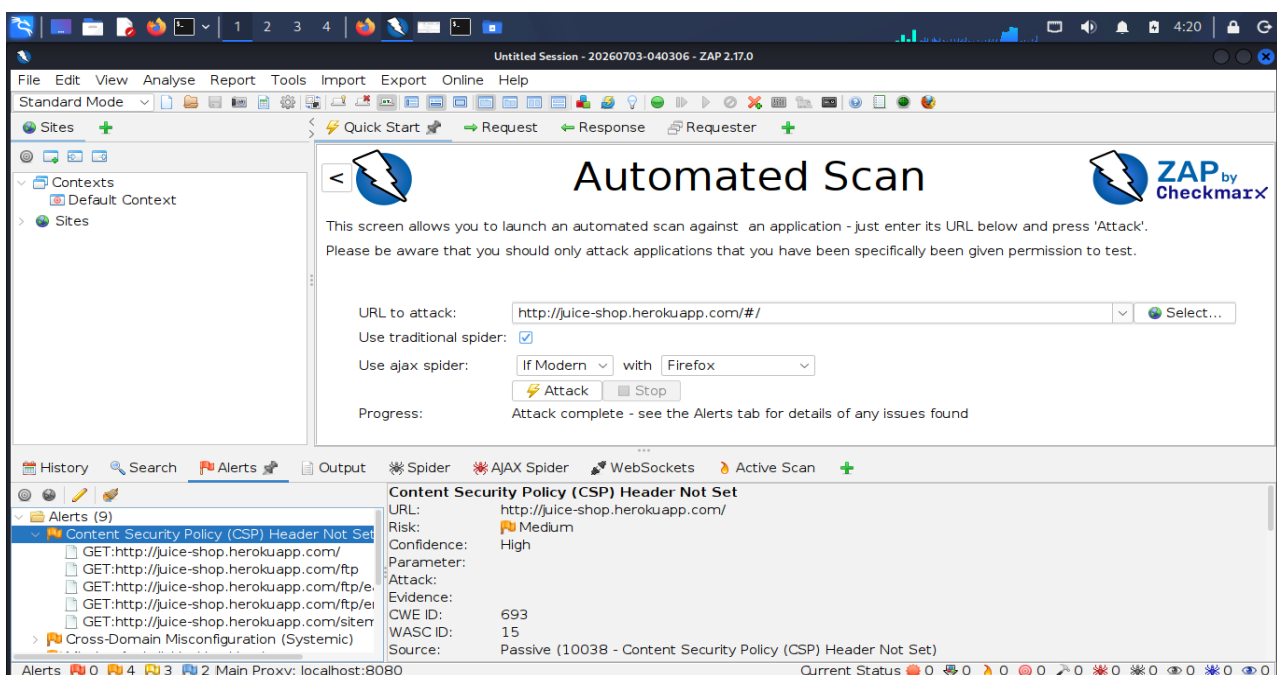
OWASP ZAP (Passive Scan)

Owasp zap (passive scan) was used to assess the web application for common security misconfigurations and information disclosure issues without actively exploiting the target. The passive scan analyzed HTTP responses and identified potential security weaknesses while ensuring the assessment remained safe and non-intrusive.

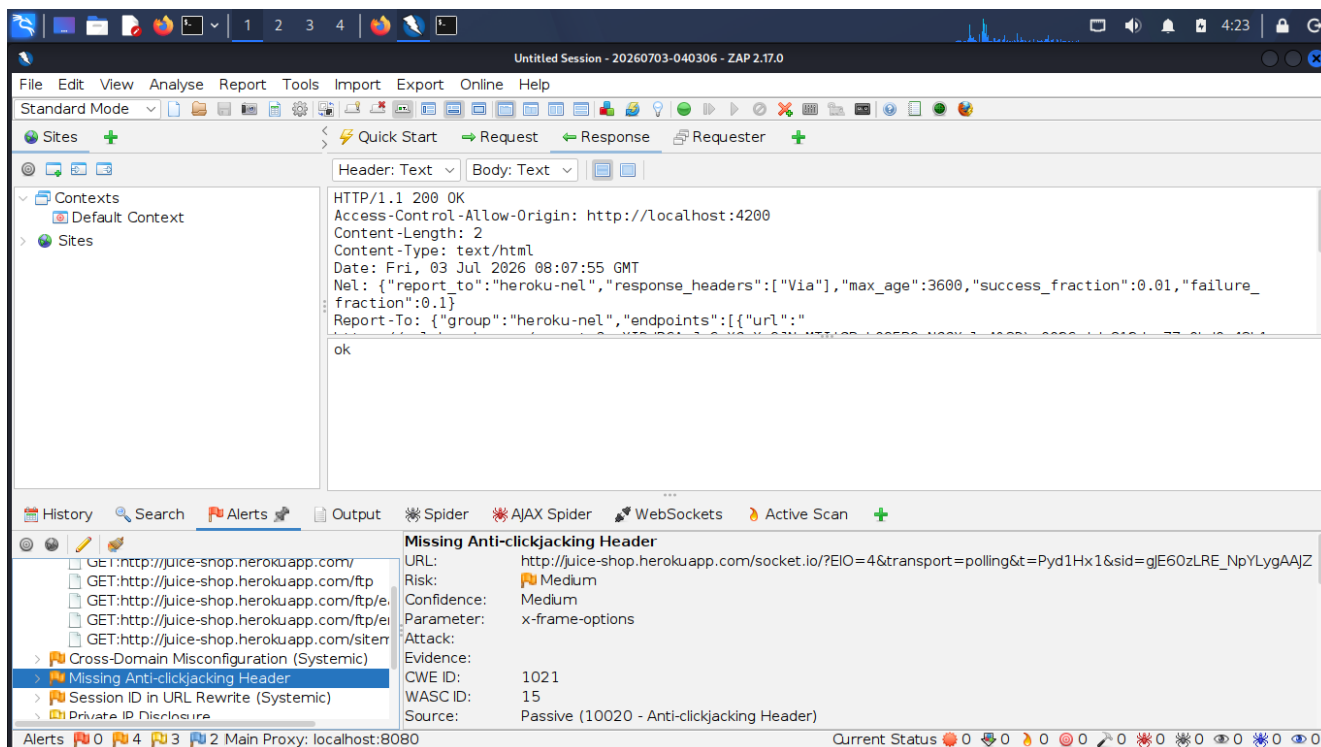
- Performed passive vulnerability analysis.
- Mapped the application's attack surface.
- Identified missing security headers and configuration issues.
- Detected information disclosure and other security-related observations.
- Classified findings based on their potential risk level.

Evidence:

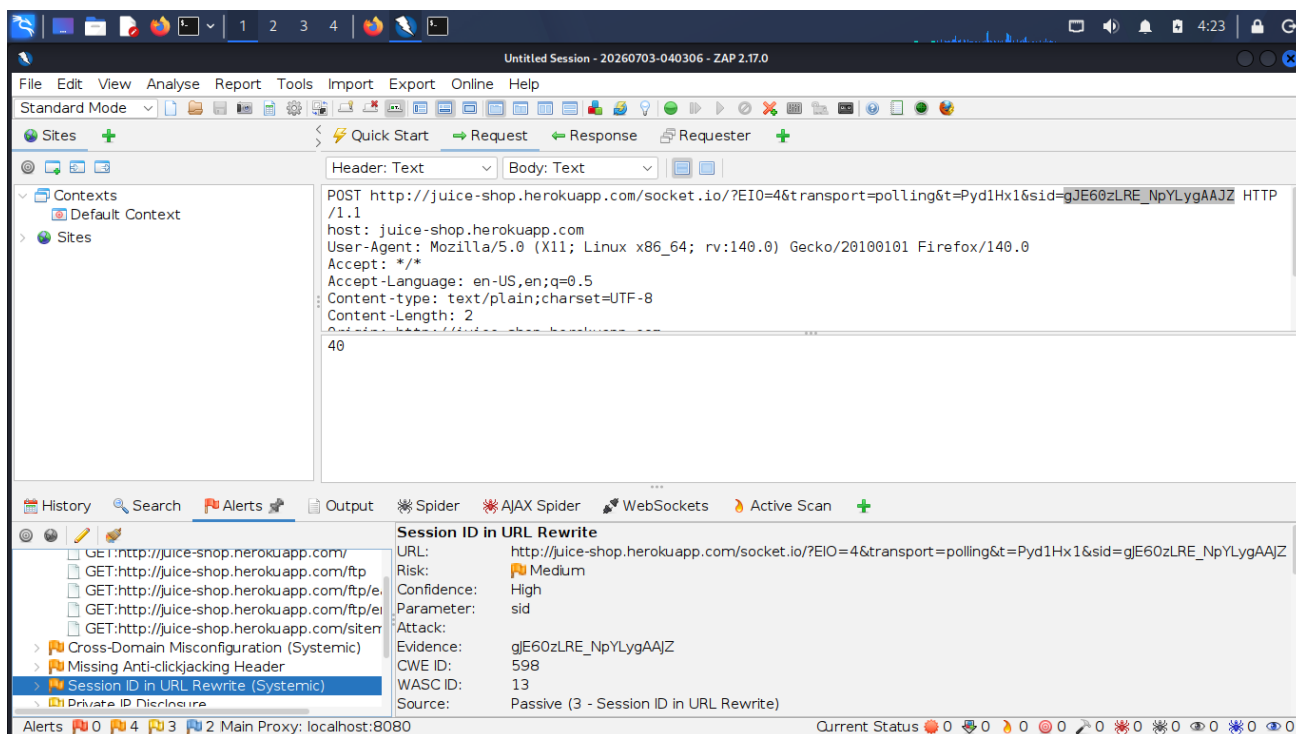
- **Figure 3:** Content Security Policy (CSP) Header Not Set



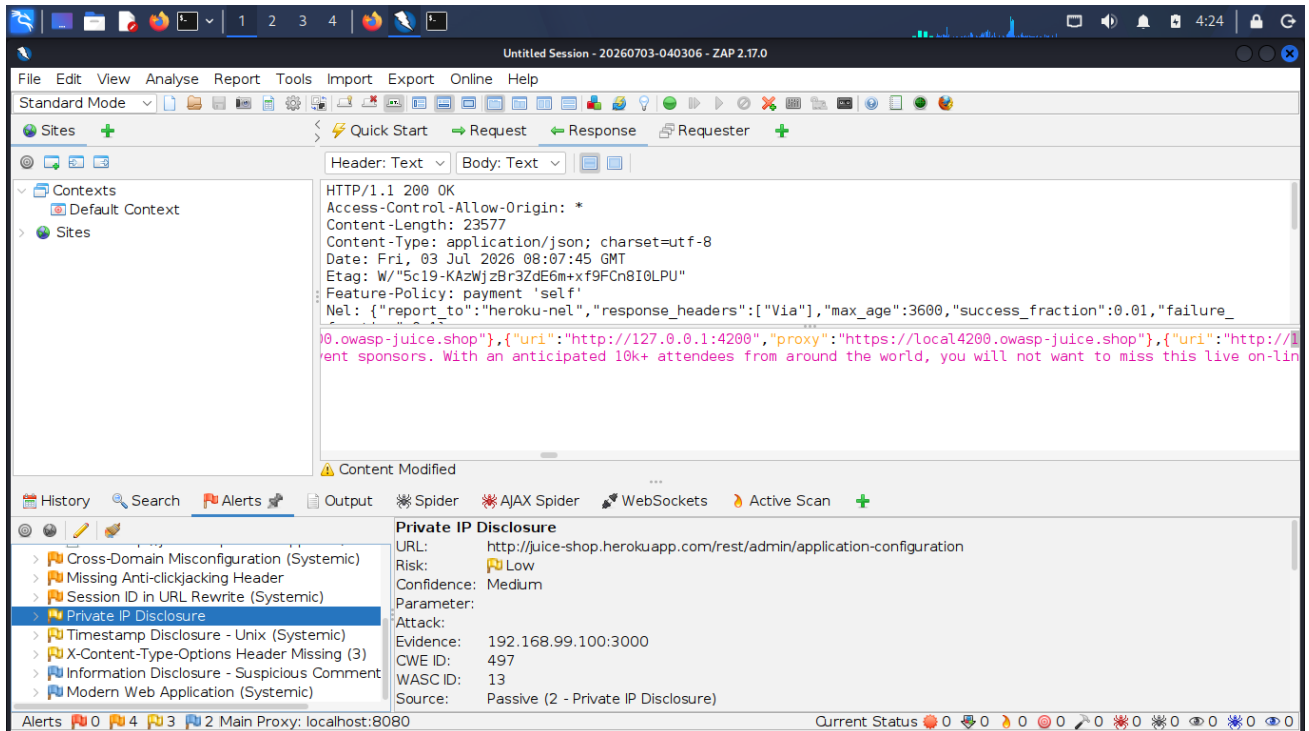
- **Figure 4: Missing Anti-Clickjacking Header**



- **Figure 5: Session ID in URL Rewrite**



- **Figure 6: Private IP Disclosure**

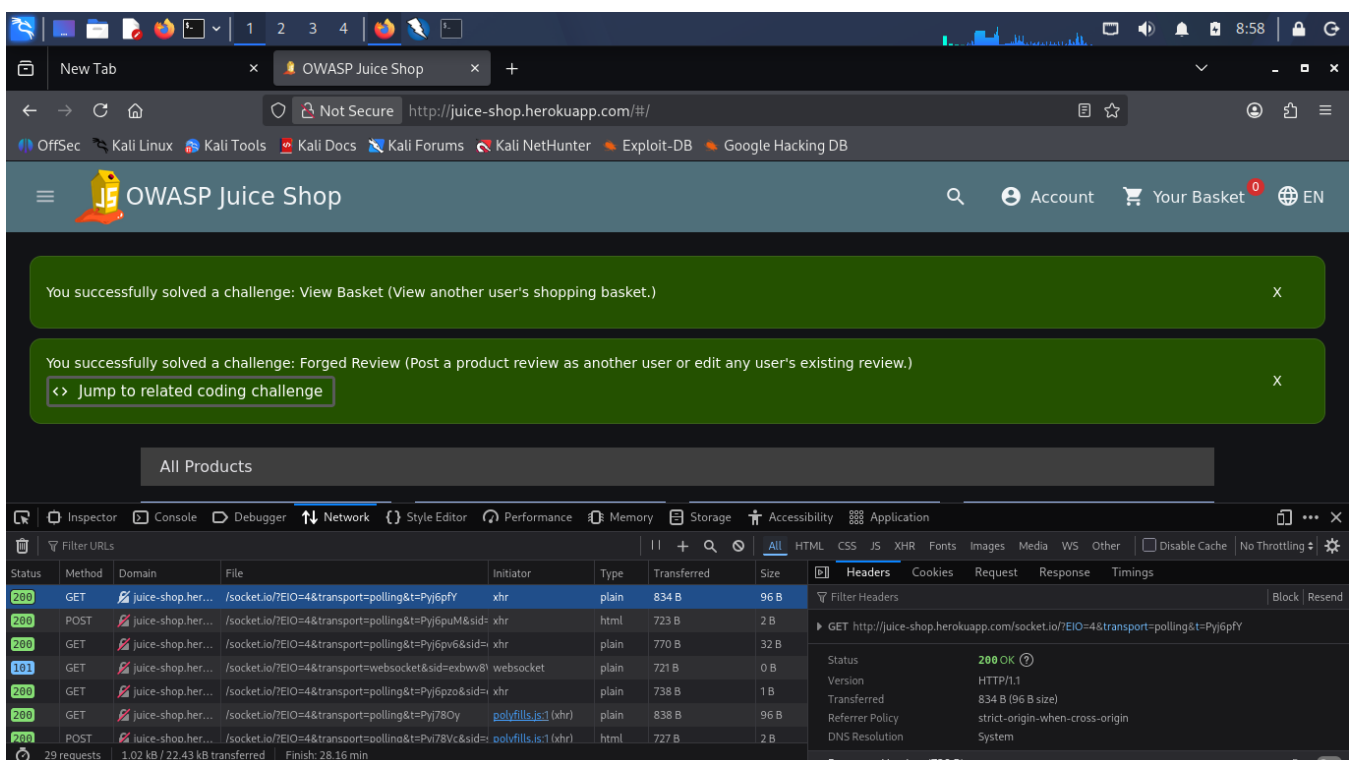


Browser Developer Tools

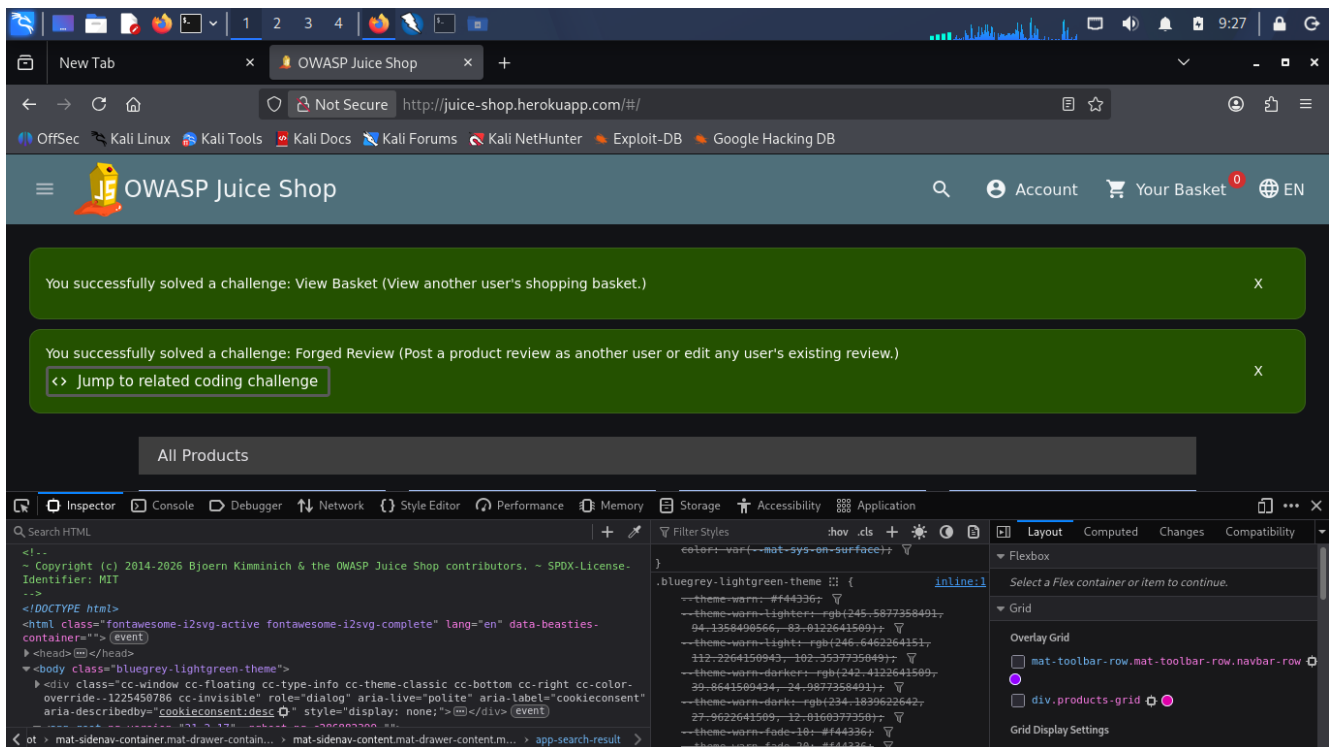
- Inspected HTTP response headers.
- Verified missing security headers.
- Reviewed cookies and network responses.
- Confirmed configuration-related findings

Evidence:

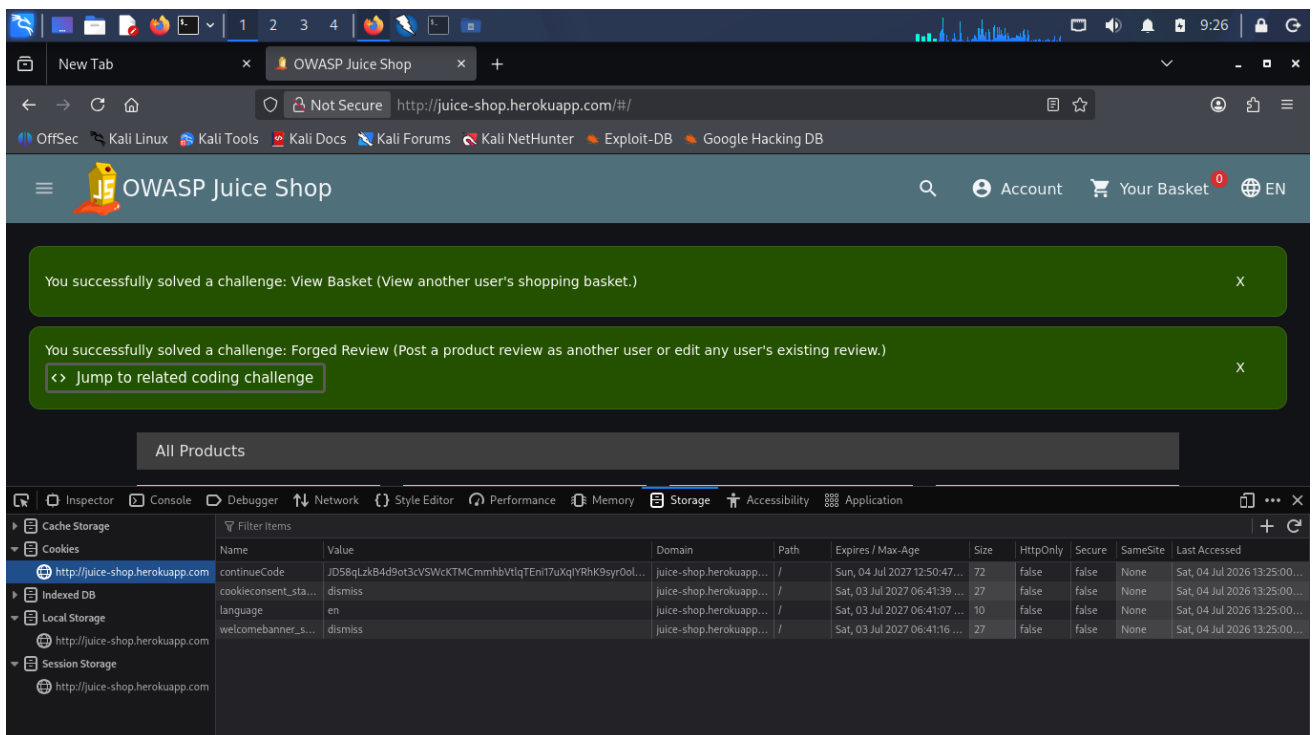
- **Figure 8: Chrome DevTools Network tab showing HTTP requests and response headers.**



- **Figure 9:** Chrome DevTools **Console** tab showing browser console logs during application execution.



- **Figure 10:** Chrome DevTools **Cookies** tab displaying session cookies and cookie attributes.



3. Findings

The assessment identified the following security issues:

- Missing **Content Security Policy (CSP)** header.
- Missing **Anti-Clickjacking** protection (X-Frame-Options or frame-ancestors).
- Session identifiers exposed through URL rewriting.
- Private IP address disclosure.
- Missing **X-Content-Type-Options** header.
- Timestamp disclosure.

- Information disclosure through application responses.
- Cross-domain misconfiguration.

4. Risk Assessment

The identified issues could affect the application's security in several ways:

- Missing **Content Security Policy (CSP)** increases the risk of Cross-Site Scripting (XSS) attacks.
- Missing **Anti-Clickjacking** protection allows attackers to embed the application inside malicious websites.
- Session identifiers in URLs may expose session information and increase the risk of session hijacking.
- Private IP address disclosure reveals internal infrastructure details that may assist attackers during reconnaissance.
- Missing security headers reduce the application's overall defense against common web-based attacks.

Risk Summary

Risk Level	Findings
Medium	Content Security Policy Header Not Set, Cross-Domain Misconfiguration, Missing Anti-Clickjacking Header, Session ID in URL Rewrite
Low	Private IP Disclosure, Timestamp Disclosure, Missing X-Content-Type-Options Header
Informational	Information Disclosure, Modern Web Application Detection

5. Remediation Recommendations

To improve the application's security posture, the following actions are recommended:

- Implement a **Content Security Policy (CSP)** to restrict trusted content sources and reduce the risk of Cross-Site Scripting attacks.
- Configure **X-Frame-Options** or the **frame-ancestors** directive within the CSP to protect against clickjacking.
- Avoid exposing session identifiers in URLs and use secure, HTTP-only session cookies instead.
- Remove internal IP addresses and sensitive information from application responses and error messages.
- Configure the **X-Content-Type-Options** header with the value nosniff to prevent MIME-type sniffing attacks.
- Review server configuration to minimize unnecessary information disclosure and implement security best practices.

6. Conclusion

The passive vulnerability assessment identified several security misconfigurations and information disclosure issues within the OWASP Juice Shop application. Although no active exploitation was performed, the identified findings should be addressed to strengthen the application's security posture and reduce the risk of future attacks. Implementing the recommended security controls will help improve resilience against common web application threats while aligning with OWASP security best practices.